

MAS Final Project: Refinement and Constraint Injection (`refinement.py`) Complete Code Documentation

1. Scope

This chapter documents `refinement.py` completely, including:

- conflict-to-constraint translation logic,
- fallback behavior when explicit conflict records are absent,
- constraint budget handling,
- integration contracts with planner and verifier.

`refinement.py` is a compact but critical module: it operationalizes the “refine” stage in the `verify` -> `refine` loop.

2. Module Imports and Dependencies

Imports:

- `Dict, List, Optional, Tuple` from `typing`
- `CooperativePlanner` from `pathfinding.py`

No external dependencies; pure transformation logic over Python data structures.

3. Public API Overview

Public function:

- `refine_planner_with_conflicts(planner, conflicts, trace=None, max_constraints=100) -> Dict[str, int]`

Private helper:

- `_add_fallback_trace_constraint(planner, trace) -> int`

Module behavior is deterministic for fixed inputs.

4. Refinement Entry Function

Signature:

- `refine_planner_with_conflicts( planner: CooperativePlanner, conflicts: List[Dict], trace: Optional[List[List[Tuple[int, int]]]] = None, max_constraints: int = 100, ) -> Dict[str, int]`

Return shape:

- `{"applied_constraints": <int>}`

This count is used by `main.py` logs.

5. High-Level Algorithm

Flow:

1. Initialize `applied_constraints = 0`.
2. If `conflicts` list is non-empty:
 - iterate conflicts in order,
 - stop when budget reached (`applied_constraints >= max_constraints`),
 - translate recognized conflict types into planner constraints.
3. Else if `trace` exists:
 - apply fallback heuristic via `_add_fallback_trace_constraint`.
4. Return applied constraint count.

Priority rule:

- explicit conflicts take precedence over trace fallback.

6. Conflict Type Mapping Rules

For each conflict dictionary:

- `conflict_type = conflict.get("type")`
- `time_step = int(conflict.get("time", 0)) + 1`

+1 shift is intentional:

- conflicts reported at simulation step `t` are converted into forbidden constraints for next planner timestep layer.

6.1 vertex Conflict

Expected payload:

- `{"type": "vertex", "pos": (x, y), ...}`

Action:

- `planner.add_constraint_position(pos, time_step)`
- `applied_constraints += 1`

6.2 edge Conflict

Expected payload:

- `{"type": "edge", "from": (x1, y1), "to": (x2, y2), ...}`

Action:

- forbid both directed edges at same time:
 - `from -> to`

- to -> from
- increments by 2.

Reason:

- prevents opposite-direction swap recurrence on that edge-time pair.

6.3 boundary Conflict

Expected payload:

- {"type": "boundary", "to": (nx, ny), ...}

Action:

- forbid that destination position at `time_step`
- increments by 1.

6.4 Ignored Types

- "separation" and unknown conflict types are intentionally ignored.

This is explicit in code comments and control flow.

7. Constraint Budget Enforcement

Budget mechanism:

- checked at top of conflict loop:
 - if already at max, break.

Important nuance:

- for edge conflicts, two constraints are added together.
- if budget threshold is crossed by edge pair, both directions are still inserted in that iteration branch.

Practical impact:

- hard cap is approximate in pair increments, but deterministic.

8. Fallback Path When No Conflicts List

Condition:

- executed only when `conflicts` is empty and `trace` is provided.

Function:

- `_add_fallback_trace_constraint(planner, trace)`

Returns integer count added (0 or 1).

Purpose:

- provide at least one coarse refinement signal when structured conflicts are unavailable.

9. Fallback Helper Detailed Logic

Signature:

- `_add_fallback_trace_constraint(planner, trace: List[List[Tuple[int, int]]]) -> int`

The `trace` format:

- outer list indexed by time,
- each item is a list of robot positions at that time.

9.1 First Pass: Exact Same-Cell Collision

For each time step from 1 to end:

1. Iterate positions at that time.
2. Track seen cells in a set.
3. If a repeated position appears:
 - add position constraint at that `time_step`,
 - return 1.

This targets explicit vertex collision recurrence first.

9.2 Second Pass: Closest Pair Heuristic at `t=1`

Used only if no exact same-cell collision found.

Guards:

- if trace length ≤ 1 -> return 0
- if fewer than 2 positions at `t=1` -> return 0

Then:

1. compute pairwise Manhattan distances among positions at time 1,
2. select pair with smallest distance,
3. choose first point of best pair as `best_position`,
4. add position constraint at time 1,
5. return 1.

If no best position found:

- return 0.

This is intentionally coarse; it biases against immediate closest-approach location.

10. Planner API Contract Required

`refinement.py` requires planner object to expose:

- `add_constraint_position(pos, t)`
- `add_constraint_edge(from_pos, to_pos, t)`

In this project, `CooperativePlanner` provides these and forwards to internal `ConstraintTable`.

11. Data Contract with Verification Output

Typical producer:

- `verification.py` via `env.last_conflicts` and counterexample trace.

Consumed fields:

- conflict dict:
 - `type`
 - `time` (optional; default 0 if absent)
 - `pos` for vertex
 - `from, to` for edge
 - `to` for boundary
- `trace`: list of robot position snapshots.

Robustness:

- missing fields are handled with guards (`is not None` checks).
- invalid/missing types are skipped without exception.

12. Interaction with Verify-Refine Loop (`main.py`)

Flow in `main.py`:

1. `verify_on_quotient(...)` returns `result`.
2. On unsafe result:
 - `conflicts = result.get("conflicts", [])`
 - `trace = result.get("counterexample")`
3. Calls:
 - `refine_planner_with_conflicts(..., max_constraints=args.refine_max_constraints)`
4. Planner is modified in place; next verification iteration sees stricter constraints.

Refinement therefore implements the corrective feedback channel.

13. Design Characteristics

1. Deterministic translation:
 - no stochastic selection.
2. Lightweight:

- no heavy search or optimization in refinement stage.
3. Incremental:
 - accumulates constraints over iterations.
 4. Conservative:
 - hard forbids can reduce feasible space and prevent repeated unsafe patterns.

14. Limits and Tradeoffs

1. Limited conflict vocabulary handling:
 - only vertex, edge, boundary are translated.
2. Separation-only unsafe signals are ignored in explicit mapping.
3. Fallback heuristic is coarse and may over-constrain localized areas.
4. Constraints are additive with no automatic pruning/expiry in this module.

These are acceptable for an engineering-first refinement loop but may require extension for broader guarantees.

15. Potential Enhancements

1. Support separation conflict translation directly (e.g., soft-to-hard local buffer constraints).
2. Add constraint metadata (source iteration, conflict id, confidence).
3. Add deduplication and optional pruning policies.
4. Introduce temporal windowing (constraint TTL) for less aggressive long-term restriction.
5. Add richer fallback using edge-level trace analysis, not only vertex coarse constraints.

16. Full Symbol Coverage Checklist

All symbols in `refinement.py` covered:

- `refine_planner_with_conflicts`
- `_add_fallback_trace_constraint`

All branches covered:

- conflicts-present path,
- budget break path,
- per-type conflict handling path,
- ignore/unknown path,
- trace fallback path,
- no-op return path.

17. Chapter Summary

`refinement.py` is the bridge from unsafe evidence to actionable planner restrictions.

It turns conflict observations into hard space-time constraints that directly affect next-iteration planning.

In this project architecture, refinement is what closes the safety loop: verification detects, refinement injects, planner adapts.